1. What is Cross-Browser Testing?

Cross-browser testing is a type of software testing that ensures a web application or website works correctly across multiple browsers, browser versions, and devices.

Different browsers (Chrome, Firefox, Safari, Edge, Opera) interpret **HTML, CSS, and JavaScript** differently, which can cause a website to display or function differently. Cross-browser testing identifies and fixes these inconsistencies.

Why is Cross-Browser Testing Important?

- Users use different browsers and devices.
- Ensure **consistent user experience**.
- Detect browser-specific bugs (layout issues, JavaScript errors).
- Improves website accessibility and usability.

---

2. Types of Cross-Browser Testing

1. **Manual Testing**
    - Open the website on multiple browsers manually and check for differences.
    - Good for small projects.
    - Time-consuming for larger apps.
2. **Automated Testing**
    - Use tools like **Selenium, BrowserStack, CrossBrowserTesting, LambdaTest** to run scripts across multiple browsers automatically.
    - Saves time and is scalable.
3. **Responsive Testing**

    - Checking how the website looks on different devices (mobile, tablet, desktop).
    - Usually overlaps with cross-browser testing because browsers on different devices render differently.

---

3. Factors to Test During Cross-Browser Testing

- **UI/UX Issues**
    - Layout differences
    - Font rendering
    - Button positions
- **Functionality Issues**
    - Forms, buttons, links
    - JavaScript features
- **Performance Issues**
    - Page load time variations
    - CSS animations
- **Compatibility Issues**
    - HTML5, CSS3, JavaScript support
    - Deprecated features in certain browsers

---

4. Example of Cross-Browser Testing

Let's say you have a **login page** with:

- Username field
- Password field
- Login button
- “Forgot password?” link

Step 1: Test in Chrome

- Open the login page
- Fill in username/password
- Click Login
- Observe behavior: works as expected

Step 2: Test in Firefox

- Open the same page
- Notice the login button appears misaligned
- JavaScript validation throws a console error
- Identify this as a **cross-browser bug**

Step 3: Test in Safari

- Password field placeholder disappears
- Submit button disabled
- Another browser-specific issue

Step 4: Test in Edge

- Works fine

Step 5: Fix Issues

- Adjust CSS for Safari/Firefox
- Fix JavaScript compatibility
- Retest all browsers

---

5. Tools for Cross-Browser Testing

Online Tools

- BrowserStack
- LambdaTest
- CrossBrowserTesting

Automation Tools

- Selenium WebDriver (for automated scripts)
- Cypress (modern JavaScript framework for testing)
- Playwright (supports multiple browsers)

Offline Tools

- Install multiple browsers locally
- Use virtual machines for older browser versions

---

6. Best Practices

1. **Prioritize Browsers**
    - Test major browsers first (Chrome, Firefox, Safari, Edge)

    - Test based on your user analytics

2. **Use a Combination**
    - Manual + Automated testing for coverage
3. **Test Early and Often**
    - Catch browser-specific issues before release
4. **Document Bugs Clearly**
    - Specify browser, version, OS, and screenshot

1. What is Ad Hoc Testing?

Definition:
Ad hoc testing is an **informal and unstructured software testing technique** where the tester tries to find defects **without following any test cases, documentation, or formal plan**.

The goal is to **break the system** by randomly interacting with it, using the tester's intuition, experience, and understanding of the application.

> Think of it as "exploring the software freely to find hidden bugs."

Key Characteristics:

- No formal test plans or test cases.
- Focus is on **finding defects quickly**.
- Done **without predefined steps**, purely exploratory.
- Usually performed by experienced testers who understand the application.

---

2. Why is Ad Hoc Testing Important?

1. **Quick detection of defects**
    - Because it's unstructured, testers may find bugs that formal testing misses.

2. **Cost-effective**
    - No test case preparation is required.
3. **Complements structured testing**
    - Formal testing may miss unexpected use cases. Ad hoc testing can catch these.
4. **Real-world scenario testing**
    - Mimics how end users interact with the system randomly.

---

3. When to Perform Ad Hoc Testing

- **After completing formal testing**: To catch any remaining defects.
- **During early development**: When formal test cases aren't ready.
- **For minor updates or quick fixes**: To ensure changes didn't break the app.

---

4. Types of Ad Hoc Testing

There are several ways to perform ad hoc testing, depending on focus:

A. Buddy Testing

- Two testers test together: one develops, the other tests simultaneously.
- Helps find defects quickly with immediate feedback.

Example:
Developer and tester check a login feature simultaneously. Tester notices validation issue while developer watches.

B. Pair Testing

- Similar to buddy testing, but focuses on **pair programming approach**.
- Tester and developer work as a pair to explore the system.

C. Exploratory Testing

- Tester explores the application **without predefined scripts**, learning the app as they go.
- Very common in agile development.

Example:
A tester checks an e-commerce site by clicking random buttons, searching products, and trying weird inputs in the search bar to see if it breaks.

D. Monkey Testing

- Random inputs are given to the system **without any knowledge of application**.
- Extreme form of ad hoc testing to find critical crashes.

Example:
Typing random text in a form field, clicking buttons rapidly, and seeing if the app crashes.

---

5. How Ad Hoc Testing is Done

1. **Understand the application basics**
    - Tester needs minimal knowledge of the application.
2. **Identify areas to test**
    - Focus on **critical modules** like login, payment, or checkout.
3. **Perform random actions**

    - Click buttons, enter random data, navigate pages, test unusual scenarios.

4. **Observe results**
    - Look for crashes, error messages, layout issues, or unexpected behavior.
5. **Report defects**
    - Log any bugs found with steps to reproduce (if possible).

---

6. Example of Ad Hoc Testing

Suppose you have a **login page**:

- Tester tries valid credentials → login works.
- Tester enters **random characters** in the username field → application crashes → defect found.
- Tester clicks “Forgot password?” repeatedly → application throws an error → defect found.
- Tester leaves password blank and presses Enter → no validation → defect found.

Here, no **test cases were followed**; the tester explored the page freely and found multiple defects.

---

7. Advantages of Ad Hoc Testing

1. **Fast and cost-effective**
    - No need for test case preparation.
2. **Finds hidden defects**
    - May catch bugs that structured testing misses.

3. **Flexible**
    - Can be performed anytime during development.
4. **Enhances tester creativity**
    - Encourages thinking outside the box.

---

8. Disadvantages of Ad Hoc Testing

1. **No documentation**
    - Makes it hard to reproduce bugs if steps are not recorded.
2. **Depends on tester skill**
    - Less experienced testers may miss defects.
3. **Not systematic**
    - Cannot guarantee full test coverage.
4. **Difficult to measure effectiveness**
    - Since there are no predefined metrics or test cases.

---

9. Best Practices for Ad Hoc Testing

1. **Know the application basics**
    - Tester should understand critical functionalities.
2. **Focus on high-risk areas**
    - Don’t randomly test everything; prioritize important features.

3. **Record bugs properly**
    - Even if testing is random, log any defects with screenshots, steps, or context.
4. **Combine with structured testing**
    - Use ad hoc testing **after or alongside formal testing** for best results.
5. **Time-box testing**
    - Set a time limit (e.g., 30 minutes of ad hoc testing per module) to keep it productive.

1. What is Exploratory Testing?

Exploratory Testing is a **software testing approach where testers actively explore the application to learn about it, design test cases, and execute tests at the same time**.

Unlike traditional testing, there are **no predefined test cases**. The tester interacts with the application, observes its behavior, and designs new tests based on what they discover.

This method focuses on **simultaneous learning, test design, and test execution**.

Simple definition:

> Exploratory testing is testing where the tester **explores the application like a real user and finds defects by experimenting with different actions.**

---

2. Key Characteristics of Exploratory Testing

1. **No predefined test cases**
2. **Test design and execution happen together**
3. **Relies on tester experience and creativity**
4. **Focus on discovering unexpected defects**
5. **Highly interactive and flexible**

---

3. Why Exploratory Testing is Important

Exploratory testing helps uncover defects that structured testing may miss.

Benefits

1. **Finds hidden bugs**
2. **Encourages critical thinking**
3. **Saves time on documentation**
4. **Good for complex systems**
5. **Useful when requirements are unclear**

Example:
A tester exploring an online shopping website might try unusual actions like:

- Adding 100 items to cart
- Applying multiple coupons
- Refreshing the payment page

These actions might reveal bugs not covered by normal test cases.

---

4. When to Use Exploratory Testing

Exploratory testing is especially useful in the following situations:

- When **requirements are incomplete or unclear**
- When **time is limited**
- During **early stages of development**
- After **formal testing is completed**

- When testing **complex applications**

---

5. How Exploratory Testing Works

Exploratory testing follows a simple cycle:

Step 1: Learn the Application

Understand how the application works.

Example:

- Learn how login, search, and checkout functions work.

Step 2: Create Test Ideas

Think of possible scenarios to test.

Example:

- Enter invalid email formats.
- Use extremely long passwords.

Step 3: Execute Tests

Perform the actions on the application.

Example:

- Enter special characters in login fields.

Step 4: Observe Results

Check if the system behaves correctly.

Example:

- If the system crashes → defect found.

Step 5: Explore Further

Use previous results to design new tests.

Example:

- If login fails incorrectly, test password reset.

---

6. Example of Exploratory Testing

Consider a **login page** with:

- Username field
- Password field
- Login button
- “Forgot password” link

A tester performs exploratory testing like this:

1. Enter correct credentials → Login successful.
2. Enter wrong password 10 times → Check if account locks.
3. Enter special characters in username → Check validation.
4. Leave fields empty → Verify error message.
5. Copy-paste extremely long text → Check system behavior.

During testing, the tester discovers:

- The system **does not show error messages properly**.

- The login page **crashes when special characters are used**.

These defects are found through exploration rather than predefined scripts.

---

7. Techniques Used in Exploratory Testing

1. Error Guessing

Tester predicts where errors might occur.

Example:
Testing boundary values like entering **0 or negative numbers** in quantity fields.

---

2. Scenario-Based Testing

Testing real-life user scenarios.

Example:
User adds item → removes item → refreshes page → checks cart.

---

3. Boundary Testing

Testing extreme values.

Example:
Password minimum length = 8 characters.

Tester checks:

- 7 characters
- 8 characters
- 50 characters

---

4. Random Testing

Entering random inputs.

Example:
Typing symbols like `@#$%^` in form fields.

---

8. Advantages of Exploratory Testing

1. **Finds critical defects quickly**
2. **Encourages tester creativity**
3. **Requires less preparation**
4. **Adapts to changes easily**
5. **Improves tester understanding of the system**

---

9. Disadvantages of Exploratory Testing

1. **Lack of documentation**
2. **Hard to reproduce defects if steps aren't recorded**
3. **Depends heavily on tester skill**
4. **Test coverage may be incomplete**

---

10. Best Practices for Exploratory Testing

1. Time-Box Testing

Limit testing sessions to a fixed time.

Example:

- 60 minutes exploring checkout functionality.

---

2. Take Notes

Record:

- Test scenarios
- Observations
- Bugs found

---

3. Focus on Critical Features

Prioritize modules like:

- Login
- Payment
- User registration

---

4. Combine with Other Testing Types

Use exploratory testing **alongside manual and automated testing**.

---

11. Real-Life Example

Testing an **online food delivery application**.

Exploratory testing actions:

1. Search for food items.
2. Add items to cart.
3. Apply discount coupon.
4. Change delivery address.
5. Cancel order after payment.

Possible bugs discovered:

- Coupon applied incorrectly.
- Payment page freezes.
- Cart resets after refresh.

---

12. Difference Between Exploratory Testing and Ad Hoc Testing

Feature	Exploratory Testing	Ad Hoc Testing
Approach	Structured exploration	Completely random
Documentation	May include notes	Usually no documentation
Planning	Some level of planning	No planning
Purpose	Learn and test simultaneously	Find bugs quickly

1. UI Validation (User Interface Validation)

Definition

UI Validation is the process of verifying that the **user interface elements of an application are displayed correctly and function properly according to the design requirements**.

UI validation ensures that the **layout, fonts, colors, buttons, images, fields, and navigation elements** appear correctly to the user.

The goal is to make sure the application is **visually correct, user-friendly, and consistent with the design specifications**.

What is Checked in UI Validation

1. Layout and Alignment

All UI elements should be properly aligned and positioned.

Example:

- Buttons should be aligned.
- Text boxes should not overlap.

Example scenario:
On a **login page**:

- Username field
- Password field
- Login button

If the **login button appears outside the form**, it is a UI issue.

---

2. Fonts and Text

Verify font style, size, color, and readability.

Example:

- Heading should be bold.
- Font should match design specifications.

Example bug:
On Chrome the text appears **normal**, but on Firefox it appears **too large**.

---

3. Colors and Themes

Colors should match the UI design.

Example:

- Submit button should be **blue**
- Cancel button should be **red**

If the cancel button appears **grey instead of red**, it is a UI defect.

---

4. Buttons and Icons

Buttons should be clickable and clearly visible.

Example:

- Hovering over the button should change color.

Bug example:
The **Add to Cart button is visible but not clickable**.

---

5. Images and Logos

Images should load properly and should not appear broken.

Example:
Company logo should display properly on the homepage.

Bug example:
Logo does not load on mobile devices.

---

6. Responsive Design

The UI should adjust properly for different screen sizes.

Example:
Website should display correctly on:

- Desktop
- Tablet
- Mobile

Bug example:
Navigation menu overlaps text on mobile screens.

Example of UI Validation

Testing an **e-commerce homepage**

Check the following:

- Logo is displayed correctly
- Navigation menu is aligned
- Product images load properly
- Buttons like **Add to Cart** are visible
- Text is readable

Possible defects:

- Button overlaps product image
- Text cut off on smaller screens
- Images appear stretched

---

2. Broken Links Testing

Definition

Broken Link Testing is the process of identifying links on a website that **do not work or lead to non-existent pages**.

A broken link occurs when a hyperlink points to a page that **no longer exists or cannot be accessed**.

Broken links usually show errors like:

- 404 Error (Page Not Found)
- 500 Error (Server Error)

Types of Links Tested

1. Internal Links

Links within the same website.

Example:
Homepage → About Us page

---

2. External Links

Links that redirect to other websites.

Example:
Company website linking to a social media page.

---

3. Image Links

Images that act as clickable links.

Example:
Clicking a banner image opens a product page.

Causes of Broken Links

- Deleted pages
- Incorrect URL
- Server issues
- Website structure changes
- Typing mistakes in links

Example of Broken Link Testing

Suppose a website has the following menu:

- Home
- About
- Services
- Contact

Tester clicks each link.

Results:

Link	Result
Home	Works
About	Works
Services	Shows 404 error
Contact	Works

Here the **Services page is a broken link**.

How to Perform Broken Link Testing

Step 1
Open the website.

Step 2
Click every link on the page.

Step 3
Verify if it redirects to the correct page.

Step 4
Check for error codes.

Tools for Broken Link Testing

Some tools automate this process:

- Screaming Frog
- Dead Link Checker
- Selenium automation scripts

- Google Search Console

---

3. Cross Validation Testing

Definition

Cross Validation Testing is the process of verifying that **data entered in one field or system is correctly validated against another field or database**.

It ensures that **related data values are consistent and accurate**.

This testing checks whether **two or more fields depend on each other correctly**.

Purpose of Cross Validation Testing

- Ensure data consistency
- Validate related fields
- Prevent incorrect data submission
- Improve data accuracy

Example 1: Date Validation

Suppose there are two fields:

Start Date
End Date

Validation rule:

End Date must be **greater than Start Date**

Test cases:

Start Date	End Date	Result
10 June	15 June	Valid
15 June	10 June	Error message
10 June	10 June	May be invalid depending on rule

If the system allows **End Date earlier than Start Date**, it is a validation defect.

Example 2: Password and Confirm Password

Fields:

Password
Confirm Password

Validation rule:

Both values must be the same.

Example:

Password: Test@123
Confirm Password: Test@123 → Valid

Password: Test@123
Confirm Password: Test@124 → Error

Example 3: Country and State Validation

If user selects:

Country = India

Then only **Indian states** should appear.

Bug example:

Country = India
State dropdown shows **California**

This indicates cross validation failure.

Example 4: Payment Validation

Fields:

Card Number
Card Expiry Date
CVV

Validation rule:

Card expiry date must be **greater than current date**.

Example bug:

Expired card still accepted.

Summary Table

Testing Type	Purpose	Example
UI Validation	Verify interface appearance	Button alignment
Broken Link Testing	Check links redirect properly	Link showing 404 error
Cross Validation Testing	Validate related data fields	Password and Confirm Password